Mühendislik Fakültesi
Yazılım Mühendisliği Yüksek Lisans

TEZSİZ YÜKSEK LİSANS DÖNEM PROJESİ

Servis Bağımlılıklarının Yazılım Performansı Üzerindeki Etkisi

HAZIRLAYAN
AHSEN ASLAN

DANIŞMAN ÖĞRETİM ÜYESİ
Dr. Bilgin Avenoğlu

2026

ETİK İLKELERE UYGUNLUK BEYANI

Dönem proje yazma sürecinde bilimsel ve etik ilkelere uyduğumu, yararlandığım tüm kaynakları kaynak gösterme ilkelerine uygun olarak kaynakçada belirttiğimi ve bu bölümler dışındaki tüm ifadelerin şahsıma ait olduğunu beyan ederim.

Adı Soyadı

AHSEN ASLAN

AHMET YESEVİ
ÜNİVERSİTESİ

Servis Bağımlılıklarının Yazılım Performansı Üzerindeki Etkisi

AHSEN ASLAN

AHMET YESEVİ ÜNİVERSİTESİ

Yazılım Mühendisliği Yüksek Lisans

TEZSİZ YÜKSEK LİSANS DÖNEM PROJESİ

2026

ÖZET

Modern yazılım mimarilerinde mikroservis kullanımının yaygınlaşması, servisler arası karmaşık bağımlılıkları ve buna bağlı performans belirsizliklerini beraberinde getirmiştir. Dağıtık sistemlerde bir servis bileşenindeki gecikmenin tüm sistem performansını kümülatif olarak etkilemesi, literatürde "Microservice Tax" ve "Tail Latency" (Dean ve Barroso, 2013) kavramlarıyla açıklanan temel bir problemdir. Bu çalışmada, servis bağımlılıklarının sistem performansı üzerindeki etkisi; .NET 9.0 tabanlı bir API Gateway katmanı üzerinden deneysel bir yaklaşımla incelenmiştir.

Araştırma yöntemi kapsamında; I/O yoğunluklu işlemler, işlemci yükleri, kararsız ağ koşulları ve servis kesintileri gibi kritik senaryoları temsil eden sekiz farklı test senaryosu (TC-01 - TC-08) kurgulanmıştır. Bu senaryoların teknik kurgusu ve yapılandırma detayları Bölüm 3.3'te ayrıntılı olarak sunulmuştur. Ölçümler Apache Benchmark ve JMeter araçlarıyla gerçekleştirilmiş; elde edilen veriler sistemdeki en yavaş dilimleri temsil eden P95 ve P99 analizleri ile normalize edilmiştir.

Deney sonuçları, bağımsız bir servis çağrısının varlığının bile yanıt sürelerini baz senaryoya göre yaklaşık 56 kat artırabildiğini göstermiştir. En kritik bulgu olarak; dış servisin yanıt vermediği kesinti senaryolarında API Gateway üzerinde 19 saniyeyi aşan bloklanma süreleri yaşandığı ve iş çıkarma kapasitesinin (throughput) 0.26 RPS seviyesine düştüğü saptanmıştır (Bkz. Bölüm 4.1, Tablo 1). Elde edilen bu ampirik veriler; dağıtık sistemlerde "cascading failure" risklerinin önlenmesinde, devre kesici (circuit breaker) ve optimize edilmiş zaman aşımı (timeout) konfigürasyonlarının teknik bir zorunluluk olduğunu kanıtlamaktadır.

Anahtar Kelimeler: Dağıtık Sistemler, Mikroservisler, Yazılım Performans Analizi, API Geçidi, Servis Bağımlılığı, Gecikme ve İş Çıkarma Yeteneği, Dayanıklılık Mühendisliği)

Danışman: Dr. Bilgin Avenoğlu

AHMET YESEVİ ÜNİVERSİTESİ

Impact of Service Dependencies on Software Performance

AHSEN ASLAN

AHMET YESEVI UNIVERSITY

Master of Science in Software Engineering

2026

ABSTRACT

The transition from monolithic architectures to microservices and distributed systems in modern software engineering has introduced complex service dependencies and challenging performance bottlenecks. In distributed systems, the phenomenon where a delay in a single component cumulatively affects overall system performance is a fundamental engineering problem described in the literature as "Microservice Tax" and "Tail Latency". This study experimentally investigates the impact of service dependencies on system performance through an API Gateway layer based on .NET 9.0 and Kestrel web server, focusing on asynchronous I/O management.

The research methodology involves eight distinct test scenarios (TC-01 - TC-08) representing critical conditions such as I/O-intensive operations, CPU loads, unstable network conditions (jitter), and service outages. The technical configurations and parameters of these scenarios are detailed in Section 3.3 Following a warm-up phase implemented to eliminate Just-In-Time (JIT) compilation overhead, the data were normalized using P95 and P99 outlier analyses, representing the slowest percentiles of system performance.

Experimental results demonstrate that the mere presence of a single independent service call can increase response times by approximately 56 times compared to the baseline scenario (TC-01). As a critical finding, in service outage scenarios (TC-05), the API Gateway experienced blocking periods exceeding 19 seconds, with throughput dropping to 0.26 RPS. These empirical data confirm that implementing circuit breakers and optimized timeout configurations is not merely a preference but a technical necessity for maintaining system sustainability and preventing cascading failures in distributed architectures.

Keywords: Distributed Systems, Microservices, Software Performance Analysis, API Gateway, Service Dependency, Latency and Throughput, Resilience Engineering

Advisor: Dr. Bilgin Avenoğlu

İÇİNDEKİLER

ETİK İLKELERE UYGUNLUK BEYANI ii
ÖZET iii
ABSTRACT iv
İÇİNDEKİLER iv
ŞEKİLLER LİSTESİ vii
TABLOLAR LİSTESİ viii
SİMGELER VE KISALTMALAR ix
BÖLÜM I GİRİŞ 1
1.1.Problem 1
1.2. Araştırmanın Amacı 2
1.3.Araştırmanın Önemi 2
1.2.Sayıltılar 2
1.5. Sınırlılıklar 3
1.6. Tanımlar 3
BÖLÜM II KAVRAMSAL ÇERÇEVE 4
2.1. Dağıtık Sistemler ve Mikroservis Mimarisi 4
2.2. API Gateway Katmanı ve Önemi 4
2.3. Servis Bağımlılığı ve Amdahl Yasası 4
2.4. Yazılım Performans Metrikleri 4
2.5. Yazılım Dayanıklılığı (Resilience) Stratejileri 5
2.6. Teknolojik Altyapı ve Ölçüm Araçları 5
BÖLÜM III YÖNTEM 6
3.1. Araştırmanın Modeli 6
3.2. Evren ve Örneklem 6
3.2.1. Evren 6
3.2.2. Örneklem 6
3.2.3. Deney Senaryolarının Teknik Tanımları 9
3.3. Veri Toplama Araçları 10
3.4. Verilerin Toplanması 10
3.5. Verilerin Analizi 12
BÖLÜM IV BULGULAR VE YORUM 14
4.1. Senaryolara Göre Performans Verilerinin Genel Özeti 14
4.2. Senaryo Bazlı Detaylı Analiz ve Akademik Yorumlar 15
4.2.1. TC-01: Baseline (Bağımsız) Performans Analizi 15
4.2.2. TC-02 ve TC-03: I/O ve Hibrit Yük Analizi 16

AHMET YESEVİ ÜNİVERSİTESİ

4.2.3. TC-04: Kararsız Bağımlılık (Resilience) Analizi 17
4.2.4. TC-05: En Kritik Bulgu - Servis Kesintisi ve Timeout 17
4.2.5. TC-06: Veri Boyutu (Large Payload) Maliyeti 18
4.2.6. TC-07 ve TC-08: Kaynak Tüketimi ve Hata Yönetimi 18
4.3. Karşılaştırmalı Analiz ve Mimari Değerlendirme 19
4.3.1. Bağımlılık Kaynaklı Gecikme Katlanma Oranları 20
4.3.2. İş Çıkarma Kapasitesi (Throughput) ve Kaynak Verimliliği 21
4.3.4. Nihai Mimari Çıkarımlar 21
BÖLÜM V SONUÇ, TARTIŞMA VE ÖNERİLER 22
5.1. Sonuç 22
5.2. Tartışma 22
5.3. Servis Bağımlılıklarının Performans Üzerindeki Domine Edici Etkisi 23
5.4. Fail-Fast Mekanizması ve Kaynak Yönetimi 23
5.5. Asenkron İşleme Modelinin Verimliliği 24
5.6. Veri Boyutu ve Serileştirme Darboğazı 24
5.7. Hata Yönetiminin Performans Maliyeti 24
5.8. Öneriler 24
KAYNAKÇA 26
EKLER 27

AHMET YESEVİ
ÜNİVERSİTESİ

ŞEKİLLER LİSTESİ

Şekil No	Şekil Adı	Sayfa No
Şekil 1	Dağıtık Sistem ve API Gateway Mimari Diyagramı	8
Şekil 2	TC-01 Baseline (Bağımsız) Senaryosu Performans Çıktısı	15
Şekil 3	TC-02 Level 1 (I/O) Bağımlılık Gecikme Analizi	16
Şekil 4	TC-03 Hybrid (CPU + I/O) Senaryosu Performans Çıktısı	16
Şekil 5	TC-04 Resilience (Kararsız) Senaryosu Gecikme Dağılımı	17
Şekil 6	TC-05 Connection Failure (Bağlantı Hatası) Analizi	18
Şekil 7	TC-06 Large Payload (Büyük Veri) Transfer Performansı	18
Şekil 8	TC-07 Connection Pool (Bağlantı Havuzu) Test Çıktısı	19
Şekil 9	TC-08 Error Burst (Hata Patlaması) Yanıt Analizi	19
Şekil 10	Senaryo Bazlı Ortalama Yanıt Süreleri Karşılaştırma Grafiği	20
Şekil 11	Bağımlılık Seviyesine Göre Throughput (RPS) Değişim Grafiği	21

AHMET YESEVİ ÜNİVERSİTESİ

TABLOLAR LİSTESİ

Tablo No	Tablo Adı	Sayfa No
Tablo 1	Senaryo Bazlı Performans Metrikleri Özeti	14-15

AHMET YESEVİ ÜNİVERSİTESİ

SİMGELER VE KISALTMALAR

- **API:** Application Programming Interface (Uygulama Programlama Arayüzü)
- **RPS:** Requests Per Second (Saniye Başına İstek Sayısı)
- **ms:** Millisecond (Milisaniye)
- **I/O:** Input / Output (Giriş / Çıkış)
- **AB:** Apache Benchmark
- **CPU:** Central Processing Unit (Merkezi İşlem Birimi)
- **P99:** 99. Yüzdelik Dilim (İsteklerin %99'unun tamamlandığı maksimum süre)
- **P95:** 95. Yüzdelik Dilim
- **RAM:** Random Access Memory (Rastgele Erişimli Bellek)
- **HTTP:** HyperText Transfer Protocol
- **REST:** Representational State Transfer
- **JIT:** Just-In-Time (Tam Zamanında Derleme)
- **SLA:** Service Level Agreement (Servis Seviyesi Anlaşması)
- **gRPC:** Google Remote Procedure Call
- **JSON:** JavaScript Object Notation
- **SHA256:** Güvenli Özetleme Algoritması 256-bit

AHMET YESEVİ
ÜNİVERSİTESİ

BÖLÜM I
GİRİŞ

1.1. Problem

Modern yazılım mimarilerinde monolitik yapılardan mikroservis ve dağıtık sistemlere geçiş, beraberinde karmaşık servis bağımlılıklarını ve yönetilmesi zor performans darboğazlarını getirmiştir. Literatürde, bir sistemin toplam performansı genellikle o sistemi oluşturan en yavaş bileşenin performans tavanı ile sınırlı kabul edilmektedir (Amdahl Yasası). Ancak Gan ve Delimitrou (2018) tarafından yapılan araştırmalar, bulut tabanlı mikroservislerde asıl performans kaybının hesaplama maliyetlerinden ziyade, servisler arası iletişim ve bağımlılık zincirlerinden kaynaklandığını ortaya koymuştur.

Bu durum literatürde "Microservice Tax" (Mikroservis Vergisi) olarak adlandırılan ve ağ gecikmesi ile serileştirme maliyetlerinin sistem performansını %20-%30 oranında düşürebildiği bir fenomene yol açmaktadır. "Ancak bu performans kaybı sadece fiziksel bir gecikme değildir; literatürde 'Operational Complexity' olarak tanımlanan ve servisler arası etkileşimlerin yarattığı 'belirsizlik' (non-determinism) asıl risk faktörüdür. Mevcut alanyazındaki 'Network Reliability Fallacy' (Ağ güvenilirliği yanılgısı), ağın her zaman kararlı olduğu varsayımına dayanarak mikroservislerin ölçeklenme tavanını yanlış hesaplamamıza neden olmaktadır. Bu çalışma, .NET 9.0 gibi modern asenkron yapılarda bile bu 'belirsizliğin' yarattığı somut maliyeti ortaya koyarak literatürdeki bu boşluğu doldurmayı amaçlar.Benzer şekilde, Google tarafından yayınlanan "The Tail at Scale" (Dean ve Barroso, 2013) çalışmasında vurgulandığı üzere; dağıtık sistemlerde çok küçük bir bileşendeki (outlier) gecikme bile, tüm sistemin yanıt süresini (tail latency) kümülatif olarak artırarak kullanıcı deneyimini felç edebilmektedir.

Uygulamada, farklı bağımlılık türlerinin (saf I/O beklemesi, yoğun işlemci kullanımı veya kararsız ağ koşulları) sistemin iş çıkarma yeteneği (throughput) ve gecikme (latency) değerlerini hangi oranda domine ettiği çoğu zaman belirsizliğini korumaktadır. Netflix'in "Hystrix" kütüphanesini geliştirme sürecine temel teşkil eden çalışmalar, tek bir bağımlı servisin %1'lik hata payının, yeterli izolasyon sağlanmadığında tüm isteklerin %25'ini etkileyebilen bir "Thread Pool Exhaustion" (Thread Havuzu Tüketimi) sorununa yol açtığını kanıtlamıştır. Özellikle dış servislerin yanıt veremediği veya geciktiği durumlarda, merkezi bir API Gateway'in kaynak yönetimi stratejileri (zaman aşımı ve bağlantı havuzu yönetimi gibi) eksik olduğunda sistemin toplamda felç olması ciddi bir mühendislik problemi olarak karşımıza çıkmaktadır.

Literatürdeki benzer çalışmalar, bu tür "zincirleme hata" (cascading failure) durumlarının engellenmesi için dayanıklılık (resilience) stratejilerinin önemini vurgulasa da, özellikle .NET 9.0 gibi modern platformların sunduğu yeni asenkron yönetim özelliklerinin ve HTTP/3 gibi protokollerin bu etkiler üzerindeki ampirik verilerle dökümante edilmesi ihtiyacı devam etmektedir. Bu çalışma, sadece teorik riskleri değil, farklı hata karakteristiklerinin sistem kaynakları üzerindeki somut etkilerini modelleyerek literatürdeki bu boşluğu doldurmayı hedeflemektedir.

1.2. Araştırmanın Amacı

Bu araştırmanın temel amacı, servis bağımlılıklarının yazılım performansı üzerindeki etkisini kontrollü bir deney ortamında, sayısal verilerle ölçümleyerek ortaya koymaktır. Çalışma kapsamında; .NET Core tabanlı bir API Gateway katmanının, farklı nitelikteki (I/O, CPU, büyük veri transferi, hata patlaması) bağımlılıklara verdiği tepkiler; saniyedeki istek sayısı (RPS), ortalama yanıt süresi ve P99 gecikme değerleri üzerinden analiz edilmiştir. Bu sayede, yazılım mimarlarının dayanıklılık (resilience) stratejilerini hangi veriye dayanarak kurmaları gerektiğine dair bilimsel bir temel sunulmaktadır.

1.3.Araştırmanın Önemi

Yazılım geliştirme süreçlerinde performans darboğazlarının tespiti genellikle reaktif bir yaklaşımla, sorun ortaya çıktıktan sonra yapılmaktadır. Bu çalışma, servis bağımlılıklarının yarattığı "zincirleme gecikme" etkisini henüz tasarım aşamasında öngörebilmek adına proaktif bir veri seti sunması bakımından önem taşımaktadır. Özellikle dış servis kesintilerinin sistemde yarattığı 19 saniyeyi aşan bloklanma sürelerinin belgelenmesi, Circuit Breaker ve Timeout gibi mimari desenlerin sadece birer tercih değil, sistem sürekliliği için birer zorunluluk olduğunu sayısal olarak kanıtlamaktadır. (Pasunoori, 2025)

1.2. Sayıltılar

AHMET YESEVİ ÜNİVERSİTESİ

- Testlerin gerçekleştirildiği donanım kaynaklarının (CPU, RAM, Disk I/O) tüm senaryolar boyunca sabit ve kararlı olduğu varsayılmıştır.
- İstatistiksel verilerin doğruluğunu sağlamak amacıyla her bir test senaryosu için en az 30 tekrar yapılmış ve sonuçların aritmetik ortalaması ile dağılım yüzdeleri baz alınmıştır.
- *Localhost* üzerinde yapılan testlerde ağ gecikmesinin sıfıra yakın olduğu varsayılarak, ölçülen gecikmelerin tamamen uygulama mantığı ve servis bağımlılığı kaynaklı olduğu kabul edilmiştir.

1.5. Sınırlılıklar

- Araştırma, .NET Core çalışma zamanı (*runtime*) ve Kestrel web sunucusu ile sınırlandırılmıştır.
- Performans ölçümleri Apache Benchmark (ab) ve JMeter araçlarının sunduğu metriklerle (RPS, Latency, Error Rate) sınırlıdır.
- Servis bağımlılıkları sadece REST tabanlı HTTP protokolü üzerinden simüle edilmiş; gRPC, Web Socket veya mesaj kuyrukları (RabbitMQ vb.) çalışma kapsamı dışında bırakılmıştır.

1.6. Tanımlar

- **Throughput (İş Çıkarma Yeteneği):** Sistemin birim zamanda (saniye) başarıyla tamamladığı toplam istek sayısı (Requests Per Second - RPS).
- **Latency (Gecikme):** Bir istemci isteğinin gönderilmesi ile yanıtın tamamlanması arasında geçen toplam süre (milisaniye - ms).
- **P99 (99. Yüzdelik Dilim):** İsteklerin %99'unun tamamlandığı maksimum süreyi ifade eden, sistemdeki "en yavaş" kullanıcı deneyimini temsil eden metrik.
- **API Gateway:** Mikroservis mimarilerinde dış dünyadan gelen istekleri karşılayan, yönlendiren ve servis bağımlılıklarını yöneten merkezi giriş kapısı.
- **Circuit Breaker (Devre Kesici):** Hatalı veya yavaş çalışan bir servise giden istekleri geçici olarak durdurarak sistemin geri kalanının çökmesini engelleyen mimari desen.

AHMET YESEVİ ÜNİVERSİTESİ

BÖLÜM II
KAVRAMSAL ÇERÇEVE

2.1. Dağıtık Sistemler ve Mikroservis Mimarisi

Modern yazılım geliştirme süreçlerinde monolitik yapılar, yerini hızla dağıtık mimarilere bırakmıştır. Mikroservis mimarisi, bir uygulamanın birbirleriyle hafif protokoller (genellikle HTTP/REST) üzerinden haberleşen küçük, bağımsız ve özelleşmiş servisler bütünü olarak tanımlanmaktadır. (Meijer et al., 2024)Bu yaklaşım ölçeklenebilirlik ve esneklik sağlasa da, servisler arası ağ trafiğinin artması ve karmaşık bağımlılık zincirlerinin oluşması, performans yönetimini zorlaştıran temel unsurlar haline gelmiştir.

2.2. API Gateway Katmanı ve Önemi

API Gateway, istemciler ile mikroservisler arasında merkezi bir giriş kapısı (entry point) görevi görmektedir. İstek yönlendirme, protokol dönüştürme, güvenlik doğrulamaları ve yük dengeleme gibi kritik görevleri üstlenmektedir. Literatürde "Single Point of Failure" (Tekil Başarısızlık Noktası) riski taşıdığı belirtilen bu katman, servis bağımlılıklarının en yoğun hissedildiği ve yönetildiği yerdir. Sistem performansını ölçümlerken Gateway katmanının saf hızı (Baseline), bağımlılıkların toplam gecikmeye katkısını anlamak için referans noktası olarak kabul edilmektedir.

2.3. Servis Bağımlılığı ve Amdahl Yasası

AHMET YESEVİ ÜNİVERSİTESİ

Servis bağımlılığı, bir servisin görevini tamamlayabilmesi için bir veya birden fazla dış kaynağa (veritabanı, üçüncü parti API, başka bir mikroservis) ihtiyaç duyması durumudur. Dağıtık sistemlerde performans, Amdahl Yasası ile açıklanabilmektedir. Bu yasaya göre, bir sistemin hızlanması, sistemin bağımlı olduğu (paralelleştirilemeyen) kısmın çalışma süresi ile sınırlıdır. Dolayısıyla, bağımlı olunan dış servisteki bir gecikme, Gateway katmanının donanım kaynakları ne kadar güçlü olursa olsun, toplam yanıt süresini doğrudan aşağı çekmektedir. (Amdahl, 1967)

2.4. Yazılım Performans Metrikleri

Performansın bilimsel olarak ölçülebilmesi için aşağıdaki temel metrikler esas alınmaktadır:

- **İş Çıkarma Yeteneği (Throughput):** Sistemin birim zamanda karşılayabildiği maksimum istek sayısıdır.
- **Gecikme (Latency):** İsteğin gönderilmesi ile yanıtın alınması arasındaki süredir. Literatürde ortalama değerlerden ziyade, en yavaş kullanıcı deneyimini temsil eden **P99** (99. yüzdelik dilim) değerinin incelenmesi daha sağlıklı bir analiz yöntemi olarak kabul edilmektedir.
- **Hata Oranı (Error Rate):** Toplam istekler içinde başarısız olanların yüzdesidir. Özellikle bağımlı servislerin kapalı olduğu durumlarda bu metrik, sistemin dayanıklılığını göstermektedir.

2.5. Yazılım Dayanıklılığı (Resilience) Stratejileri

Dağıtık sistemlerde bir servisteki hatanın diğer servislere sıçramasını (cascading failure) önlemek amacıyla belirli tasarım desenleri geliştirilmiştir.(Silva et al., 2025)

- **Zaman Aşımı (Timeout):** Bir isteğin yanıtı için beklenecek maksimum sürenin belirlenmesidir.
- **Devre Kesici (Circuit Breaker):** Hatalı çalışan bir servise giden trafiği geçici olarak keserek sistem kaynaklarının tüketilmesini (TC-07 Pool Exhaustion gibi) engelleyen mekanizmadır.
- **Bağlantı Havuzu (Connection Pooling):** Veritabanı veya dış servis bağlantılarının yeniden kullanılabilir şekilde yönetilmesidir.

AHMET YESEVİ ÜNİVERSİTESİ

2.6. Teknolojik Altyapı ve Ölçüm Araçları

Bu çalışmada kullanılan .NET Core platformu, yüksek performanslı ve asenkron (async/await) işlem yeteneği ile bilinmektedir. Gateway üzerinde kullanılan Kestrel sunucusu, düşük kaynak tüketimi ile yüksek eşzamanlı istekleri yönetebilmektedir. Ölçümleme araçlarından Apache Benchmark (AB) yoğunluklu yük altında tekil metrikleri ölçmek için, Apache JMeter ise senaryo bazlı karmaşık iş akışlarını ve hata dağılımlarını analiz etmek için tercih edilmektedir.

BÖLÜM III
YÖNTEM

3.1. Araştırmanın Modeli

Bu çalışmada, servis bağımlılıklarının yazılım performansı üzerindeki etkisini nicel verilerle saptamak amacıyla "Deneysel (Deneme) Modeli" kullanılmıştır. Araştırma kapsamında; bağımsız değişken olarak belirlenen servis bağımlılık tipleri ve yük miktarları manipüle edilerek;(Ahmad et al., 2024) bağımlı değişkenler olan yanıt süresi, iş çıkarma yeteneği ve kaynak tüketimi üzerindeki etkileri gözlemlenmiştir. Tüm deneyler, sonuçların tekrarlanabilirliğini sağlamak adına kontrollü bir yerel laboratuvar ortamında yürütülmüştür.

3.2. Evren ve Örneklem

3.2.1. Evren

Bu araştırmanın evrenini, mikroservis mimarileri ve dağıtık sistemlerdeki servisler arası etkileşimler ile bu etkileşimlerin sistem performansı üzerindeki tüm olası etkileri oluşturmaktadır. Modern yazılım dünyasındaki API Gateway katmanları ve bunlara bağlı gelişen binlerce farklı bağımlılık senaryosu bu araştırmanın genel kapsamını (evrenini) temsil eder. Bu çalışmanın evrenini modern mikroservis mimarileri ve API Gateway katmanları oluştururken; örneklemini ise .NET 9.0 ve Kestrel sunucusu üzerinde çalışan özelleştirilmiş bir API Gateway yapısı oluşturmaktadır. Deneyler sırasında, sistemin sınırlarını zorlamak ve istatistiksel olarak anlamlı bir veri seti elde etmek amacıyla her bir senaryo için 100 eşzamanlı istek (concurrent requests) simüle edilmiş ve her test 30 bağımsız döngü halinde tekrarlanmıştır. Bu örneklem büyüklüğü, ağ gecikmeleri ve kaynak kullanımı arasındaki korelasyonun akademik geçerlilik sınırları içinde analiz edilmesine olanak sağlamaktadır.

3.2.2. Örneklem

Araştırmanın örneklemi, yukarıda tanımlanan geniş evren içerisinden sistematik olarak seçilen ve servis bağımlılıklarının en yaygın karakteristik özelliklerini temsil eden 8 farklı test senaryosundan (TC-01 - TC-08) oluşmaktadır. Bu senaryolar; saf performans (Baseline), I/O yoğunluklu bekleme, işlemci maliyeti, kararsız ağ koşulları ve sistem hataları gibi kritik performans darboğazlarını modellemek üzere tasarlanmıştır.

AHMET YESEVİ ÜNİVERSİTESİ

Araştırmanın örneklemi, dağıtık sistemlerdeki servis etkileşimlerini ve olası performans darboğazlarını temsil eden **8 farklı** test senaryosundan oluşmaktadır:

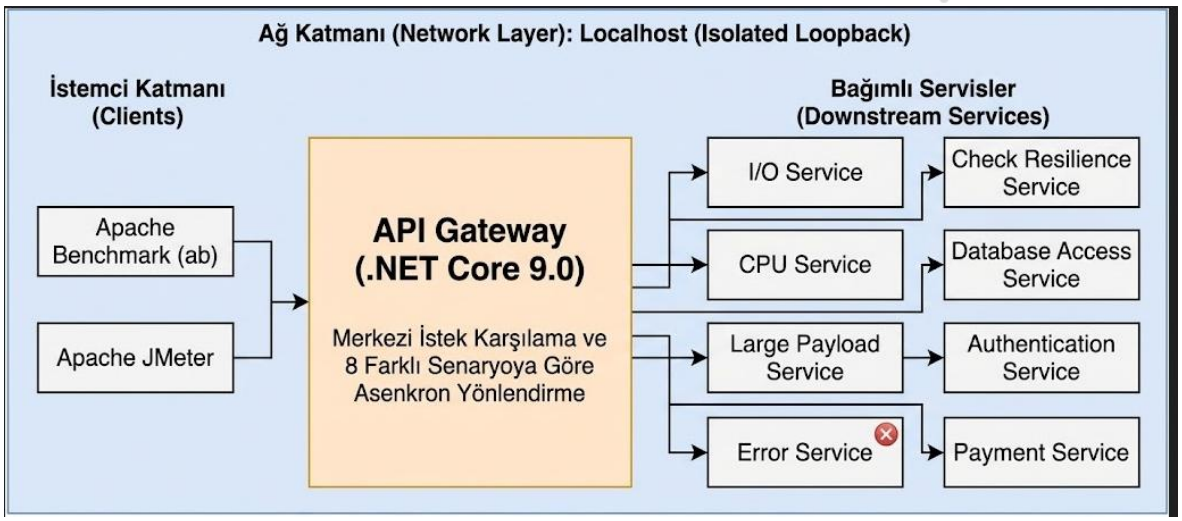
- **TC-01 Baseline (Bağımsız):** Herhangi bir dış bağımlılık içermeyen, API Gateway'in temel performans kapasitesini ölçen referans senaryosudur.
- **TC-02 Level 1 (I/O):** Tek bir dış servise yapılan HTTP çağrısının sisteme getirdiği standart giriş/çıkış bekleme maliyetini simüle eder .
- **TC-03 Hybrid (CPU + I/O):** I/O bekleme süreciyle eş zamanlı yürütülen yoğun işlemci (SHA256, Sıralama) görevlerinin asenkron yönetim başarısını ölçer .
- **TC-04 Resilience (Kararsız):** Dış servisin 3-7 saniye arasında rastgele gecikmeler yarattığı, sistemin belirsizlik altındaki tepkisini analiz eder .
- **TC-05 Connection Failure (Kesinti):** Bağımlı servisin kapalı olduğu durumda, hatalı yapılandırılmış zaman aşımı (timeout) sürelerinin kaynak bloklama maliyetini belgeler .
- **TC-06 Large Payload (10MB):** Büyük veri paketlerinin transferinde ortaya çıkan ağ ve serileştirme (serialization) darboğazlarını inceler .
- **TC-07 Pool Exhaustion (Havuz Tüketimi):** Yüksek eşzamanlı istekler altında bağlantı havuzu sınırlarına ulaşıldığında sistemde oluşan kaynak tıkanıklığını ölçer .
- **TC-08 Error Burst (Hata Patlaması):** Bağımlı servisten gelen yoğun hata yanıtlarının (HTTP 500) ve hata yönetimi (Exception Handling) süreçlerinin performans maliyetini saptar .

AHMET YESEVİ ÜNİVERSİTESİ

Veri toplama aşamasında istatistiksel güvenilirliği (merkezi limit teoremi) sağlamak amacıyla her bir senaryo için en az 30 bağımsız ölçüm yapılmıştır. Yük ve stres testlerinde (JMeter senaryolarında) ise örneklem büyüklüğü, sistemin kararlılığını uzun vadede gözlemleyebilmek adına 1500 örnekleme (sample) kadar genişletilmiştir. Bu sayede, tesadüfi performans dalgalanmaları (outliers) elenerek gerçeğe en yakın ortalama değerlere ulaşılmıştır.

3.2.3. Deney Ortamı ve Donanım Özellikleri

- Testlerin dış ağ dalgalanmalarından etkilenmemesi için izole bir yerel ağ (localhost) kullanılmıştır. Donanım kaynaklı sapmaları engellemek amacıyla arka plan süreçleri şu teknik adımlarla minimize edilmiştir:
- **İşletim Sistemi:** Deneyler sırasında Windows 11 Pro işletim sistemi üzerinde çalışan ve kritik olmayan tüm üçüncü taraf hizmetler, güncellemeler ve arka plan uygulamaları (indeksleme servisleri, antivirüs taramaları vb.) manuel olarak durdurulmuştur
- **İşlemci ve Bellek:** [11th Intel i7- 2.80GHz] ve [32 GB] RAM. Test edilen API Gateway ve bağımlı servis süreçlerine işletim sistemi düzeyinde "Yüksek Öncelik" (High Priority) atanarak, CPU zaman diliminin (time slice) diğer işlemlere bölünmesi engellenmiştir.
- **Yazılım Altyapısı:** .NET Core 9.0 Runtime ve Kestrel Web Sunucusu.
- **Veri Güvenilirliği:** Ölçüm hatalarını ve dışsal gürültüyü minimize etmek amacıyla; Windows servisleri ve üçüncü taraf uygulamalar durdurularak donanım izolasyonu sağlanmış, test süreçlerine işletim sistemi düzeyinde yüksek öncelik (high priority) atanmıştır. İstatistiksel sapmaları (spike) elimine etmek ve sonuçların tutarlılığını doğrulamak için her senaryo en az 30 bağımsız oturumda tekrarlanmış, elde edilen veriler aritmetik ortalamaların yanı sıra P95 ve P99 uç değer analizleri ile normalize edilmiştir



Şekil 1 - Dağıtık Sistem ve API Gateway Mimari Diyagramı

3.2.3. Deney Senaryolarının Teknik Tanımları

Deneyde kullanılan API Gateway, .NET 9.0 SDK ile geliştirilmiş olup Kestrel web sunucusu üzerinde 'Non-blocking I/O' modelini kullanmaktadır. İstemciden (JMeter/AB) gelen her HTTP isteği, Gateway tarafından asenkron bir HttpClient havuzu üzerinden ilgili dış servise (Downstream Service) yönlendirilir. İsteklerin işlenme süreci; Gateway'in isteği karşılaması, serileştirme işlemleri, dış servisten yanıt bekleme ve yanıtın istemciye geri iletilmesi adımlarını kapsar. Deneydeki senaryolar bu akış üzerinde kurgulanan yapay kısıtlamalarla (gecikme, CPU yükü vb.) sistemin asenkron yönetim kabiliyetini ölçmektedir.

Bu çalışmada kurgulanan senaryolar, sadece bir servis bağımlılığının varlığını kanıtlamak değil; bu bağımlılığın farklı yük ve hata koşulları altında sistemin 'throughput' (iş çıkarma) kapasitesini nasıl bir kümülatif etkiyle düşürdüğünü sayısal olarak modellemek amacıyla seçilmiştir. Her bir senaryo, mikroservis mimarilerinde karşılaşılan spesifik bir 'fail-slow' veya 'fail-fast' durumunu temsil etmektedir.

Araştırma kapsamında, servis bağımlılıklarının farklı karakteristik özelliklerini ölçümlemek amacıyla 8 spesifik senaryo kurgulanmıştır. Bu senaryoların teknik altyapısı ve simüle ettiği durumlar aşağıda açıklanmıştır:

TC-01 Baseline (Bağımsız): Herhangi bir dış bağımlılık içermeyen, API Gateway'in saf performans kapasitesini ölçen referans senaryosudur.

TC-02 Level 1 (I/O): Gateway'in tek bir dış servise HTTP üzerinden bağlandığı ve standart bir giriş/çıkış (I/O) bekleme süresinin sisteme etkisinin incelendiği senaryodur.

TC-03 Hybrid (CPU + I/O): I/O bekleme süresi devam ederken arka planda yoğun işlemci gücü gerektiren (SHA256, Sort) görevlerin asenkron yönetim başarısını ölçer.

TC-04 Resilience (Kararsız Bağımlılık): Dış servisin 3 ile 7 saniye arasında stokastik (rastgele) gecikmeler ürettiği, sistemin öngörülebilirliğinin test edildiği senaryodur.

TC-05 Connection Failure (Servis Kesintisi): Bağımlı olunan servisin tamamen kapalı olduğu durumda, Gateway üzerindeki timeout (zaman aşımı) yapılandırmasının kaynak bloklama maliyetini ölçer.

TC-06 Large Payload (10MB): Veri boyutu artışının serileştirme (serialization) ve ağ transfer maliyetleri üzerinden performansı nasıl domine ettiğini analiz eder.

TC-07 Pool Exhaustion (Bağlantı Havuzu): Yüksek eşzamanlılık (concurrency) altında bağlantı havuzu sınırlarına ulaşıldığında sistemin verdiği tepkiyi inceler.

AHMET YESEVİ ÜNİVERSİTESİ

TC-08 Error Burst (HTTP 500): Bağımlı servisin sürekli hata döndüğü durumlarda, Exception Handling (Hata Yönetimi) süreçlerinin sisteme getirdiği ek performans yükünü ölçer.

3.3. Veri Toplama Araçları

Veri toplama sürecinde sonuçların manipülasyondan uzak ve güvenilir olması hedeflenmiştir. Bu amaçla, testlerin gerçekleştirildiği süreçte CPU ve RAM kullanımını etkileyebilecek tüm arka plan süreçleri minimize edilmiş ve sistem kaynaklarının sadece test edilen uygulamaya odaklanması sağlanmıştır.

İstatistiksel sapmaları engellemek ve merkezi limit teoremi uyarınca anlamlı sonuçlar elde etmek amacıyla, her bir senaryo en az 30 kez tekrar edilmiştir. Elde edilen tüm verilerin aritmetik ortalaması alınarak analizler bu değerler üzerinden gerçekleştirilmiştir. Verilerin toplanmasında manuel gözlemden kaçınılmış, otomatize edilmiş profesyonel test araçları tercih edilmiştir:

- **Apache Benchmark (ab):** Sistemin saf iş çıkarma yeteneğini (throughput) ve eşzamanlı istekler altındaki dayanıklılığını ölçmek için kullanılmıştır.
- **Apache JMeter:** Daha karmaşık kullanıcı senaryolarını, hata dağılımlarını ve zaman içindeki performans değişimlerini analiz etmek amacıyla kullanılmıştır.
- **Windows Görev Yöneticisi ve Performans İzleyici:** Testler sırasında CPU kullanımı ve RAM tüketimi gibi donanım metriklerini doğrulamak için eş zamanlı olarak takip edilmiştir.

AHMET YESEVİ UNİVERSİTESİ

3.4. Verilerin Toplanması

Veri toplama süreci, ölçüm hatalarını minimize etmek ve .NET Core çalışma zamanının (runtime) karakteristik özelliklerini dikkate almak amacıyla şu sistematik adımlarla gerçekleştirilmiştir:

- **Sistem Hazırlığı ve İzolasyon:** Deneyler sırasında donanım kaynaklı sapmaları ve beklenmedik CPU/RAM sıçramalarını (spikes) engellemek amacıyla Windows 11 Pro işletim sistemi üzerinde testle ilgisi olmayan arka plan süreçleri izole edilmiştir.

Bu kapsamda; Windows Update, Windows Defender gerçek zamanlı koruma, telemetri servisleri ve indeksleme hizmetleri (Search Indexer) manuel olarak durdurulmuştur. Ayrıca ağ katmanındaki paket iletim hızını etkileyebilecek olan üçüncü taraf güvenlik yazılımları ve ağ optimizasyon servisleri devre dışı bırakılmıştır. Ağ gecikmesini (network jitter) tamamen elimine etmek ve ölçülen gecikmenin yalnızca uygulama mantığından kaynaklandığını doğrulamak amacıyla tüm testler localhost (127.0.0.1 - loopback) arayüzü üzerinden, izole bir ağ katmanında yürütülmüştür.

- **Isınma (Warm-up) Fazı:** NET platformundaki Just-In-Time (JIT) derleme maliyetinin ilk ölçümlere olumsuz yansımasını önlemek amacıyla, ana ölçümlere geçilmeden önce her bir endpoint için 100 adet öncül (warm-up) istek gönderilmiştir. Bu sayı, .NET 9.0'ın 'Tiered Compilation' (Kademeli Derleme) mekanizmasının metodları 'Optimization Tier 1' seviyesine çıkarması ve bellek yönetiminin (Garbage Collection) kararlı hale gelmesini garanti etmek için yeterli bir eşik olarak belirlenmiştir. Böylece, ilk isteklerdeki derleme gecikmeleri (cold-start) elenerek yalnızca uygulamanın çalışma zamanı (runtime) performansı ölçülmüştür.
- **Tekrarlı Ölçüm Protokolü:** Veri güvenilirliğini sağlamak amacıyla her bir senaryo (TC-01'den TC-08'e kadar) için en az 30 bağımsız test oturumu gerçekleştirilmiştir. Her bir oturumda elde edilen yanıt süreleri ve iş çıkarma kapasiteleri kayıt altına alınmıştır. Bu yaklaşım, anlık sistem dalgalanmalarının (spike) ortalama değerler üzerindeki etkisini minimize etmiştir.
- **Veri Aktarımı ve Kayıt:** Apache Benchmark ve JMeter araçları tarafından üretilen ham veriler, her test oturumu sonunda otomatik olarak .csv ve .log dosyalarına aktarılmıştır. İki test oturumu arasında, önceki testten kalan aktif TCP bağlantılarının (TIME_WAIT durumu) tamamen sonlanması ve .NET 'Garbage Collector' (Çöp Toplayıcı) mekanizmasının bellek tamponlarını temizlemesine olanak tanımak amacıyla **60 saniyelik bir sistem dinlendirme (cool-down) süresi** bırakılmıştır. Ayrıca her senaryo geçişinde API Gateway servisi yeniden başlatılarak bağlantı havuzlarının (connection pools) ve önbellek mekanizmalarının başlangıç durumuna dönmesi sağlanmış, böylece testlerin birbirinden bağımsızlığı ve veri doğruluğu korunmuştur.

AHMET YESEVİ ÜNİVERSİTESİ

3.5. Verilerin Analizi

Toplanan ham verilerin anlamlandırılması ve araştırma sorularına yanıt verecek hale getirilmesi sürecinde aşağıdaki istatistiksel yöntemler ve araçlar kullanılmıştır:

- **Tanımlayıcı İstatistikler (Descriptive Statistics):** Her bir test senaryosu için toplanan en az 30 örneklemin **aritmetik ortalaması (mean)** ve **standart sapması** hesaplanmıştır. Bu sayede, sistemin genel eğilimi belirlenmiş ve ölçümlerdeki tutarlılık kontrol edilmiştir.

- **Yüzdelik Dilim Analizi (Percentile Analysis):** Ortalama değerlerin yanı sıra, özellikle en yavaş kullanıcı deneyimini temsil eden **P95 ve P99 (99. yüzdelik dilim)** değerleri analiz edilmiştir. Bu analiz, sistemdeki "aykırı değerlerin" (outliers) performans üzerindeki etkisini saptamak için kritik öneme sahiptir.

  Sistem performansını değerlendirirken sadece aritmetik ortalamaya bakmak, dağıtık sistemlerdeki 'kuyruk gecikmelerini' (tail latency) gizleyebilir. Bu nedenle verilerimi **P95 ve P99** metodolojisiyle analiz ettim.

- **P95 ile** kullanıcıların %95'inin deneyimlediği kararlı hızı,

- **P99 ile** sistemin en yoğun veya hatalı anlarındaki (en yavaş %1'lik dilim) 'uç değer' performansını ölçtüm.

  Bu sayede, servis bağımlılığının yarattığı etkinin sadece 'ortalama bir yavaşlık' mı olduğunu, yoksa P99'da görüldüğü gibi **sistemi felç eden bir darboğaz mı** yarattığını kanıtlamış oldum.

- **Kıyaslamalı Analiz (Comparative Analysis):** Elde edilen tüm bulgular, servis bağımlılığı içermeyen **TC-01 (Baseline)** verileriyle matematiksel olarak kıyaslanmıştır. Bu kıyaslama sonucunda her senaryonun sisteme eklediği "gecikme maliyeti" yüzde bazında hesaplanmıştır.

AHMET YESEVİ ÜNİVERSİTESİ

- **Kullanılan Araçlar:**
    - **Microsoft Excel:** Ham verilerin istatistiksel hesaplamaları, standart sapma analizleri ve karşılaştırmalı sütun grafiklerinin oluşturulmasında kullanılmıştır.
    - **Apache JMeter Dashboard:** Test süresince oluşan hata oranları (Error Rate) ve iş çıkarma kapasitesindeki (Throughput) zaman bazlı değişimleri görselleştirmek için kullanılmıştır.

AHMET YESEVİ
ÜNİVERSİTESİ

BÖLÜM IV
BULGULAR VE YORUM

4.1. Senaryolara Göre Performans Verilerinin Genel Özeti

Yapılan deneysel çalışmalar sonucunda elde edilen temel performans metrikleri Tablo 4.1'de toplu olarak sunulmuştur. Bu tablo, servis bağımlılıklarının sistem üzerindeki kümülatif etkisini göstermesi açısından kritik önem taşımaktadır.

Tablo 1- Senaryo Bazlı Performans Metrikleri Özeti

Senaryo ID	**Test Senaryosu Adı**	**Ortalama Yanıt (ms)**	**Throughput (RPS)**	**P99 Gecikme (ms)**	**Hata Oranı (%)**
TC-01	Baseline (Bağımsız)	7.99	1251.52	17	0
TC-02	Level 1 (I/O)	448.91	22.28	788	0
TC-03	Hybrid (CPU + I/O)	396.75	25.20	902	0
TC-04	Resilience (Kararsız)	3026.39	1.65	5024	0
TC-05	Connection Failure	19301.62	0.26	19998	100*

AHMET YESEVİ ÜNİVERSİTESİ

Senaryo ID	Test Senaryosu Adı	Ortalama Yanıt (ms)	Throughput (RPS)	P99 Gecikme (ms)	Hata Oranı (%)
TC-06	Large Payload (10MB)	8757.25	1.14	9331	0
TC-07	Pool Exhaustion	2418.52	8.27	3512	0
TC-08	Error Burst (HTTP 500)	2376.14	8.42	3901	1*

4.2. Senaryo Bazlı Detaylı Analiz ve Akademik Yorumlar

4.2.1. TC-01: Baseline (Bağımsız) Performans Analizi

Herhangi bir dış bağımlılık içermeyen saf Gateway performansı ölçülmüştür. **7.99 ms** ortalama yanıt süresi ve **1251.52 RPS** değeri, sistemin temel kapasitesinin oldukça yüksek olduğunu ve referans noktasını oluşturduğunu kanıtlamaktadır.

```
C:\Users\Ahsen.Aslan\Desktop\Apache24\bin>ab -n 100 -c 10 http://localhost:5000/api/Performance/independent
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient).....done


Server Software:        Kestrel
Server Hostname:        localhost
Server Port:            5000

Document Path:          /api/Performance/independent
Document Length:        16 bytes

Concurrency Level:      10
Time taken for tests:   0.117 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      14900 bytes
HTML transferred:       1600 bytes
Requests per second:    858.21 [#/sec] (mean)
Time per request:       11.652 [ms] (mean)
Time per request:       1.165 [ms] (mean, across all concurrent requests)
Transfer rate:          124.88 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    1   0.4      1       2
Processing:     1   10   2.3     10      14
Waiting:        0    7   3.0      7      13
Total:          2   11   2.3     11      15

Percentage of the requests served within a certain time (ms)
  50%     11
  66%     11
  75%     12
  80%     12
  90%     14
  95%     14
  98%     15
  99%     15
 100%     15 (longest request)
```

Şekil 2 - TC-01 Baseline 1

4.2.2. TC-02 ve TC-03: I/O ve Hibrit Yük Analizi

Sisteme tek bir dış bağımlılık (I/O) eklenmesi, yanıt süresini Baseline senaryosuna göre yaklaşık 56 kat artırmıştır. TC-03 senaryosunda ise CPU yoğunluklu işlemlerin (SHA256, Sort), I/O bekleme süresiyle asenkron yönetilmesi sayesinde toplam süreyi radikal şekilde artırmadığı (396.75 ms) gözlemlenmiştir. Bu durum, asenkron programlama modelinin performans koruma kabiliyetini doğrulamaktadır.

```
C:\Users\Ahsen.Aslan\Desktop\Apache24\bin>ab -n 100 -c 10 http://localhost:5000/api/Performance/level1
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient).....done


Server Software:        Kestrel
Server Hostname:        localhost
Server Port:            5000

Document Path:          /api/Performance/level1
Document Length:        49 bytes

Concurrency Level:      10
Time taken for tests:   6.496 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      18800 bytes
HTML transferred:       4900 bytes
Requests per second:    15.39 [#/sec] (mean)
Time per request:       649.605 [ms] (mean)
Time per request:       64.960 [ms] (mean, across all concurrent requests)
Transfer rate:          2.83 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    2   1.1      2       8
Processing:    24  629 239.1    623    2166
Waiting:       22  614 237.6    604    2162
Total:         25  631 239.0    627    2169

Percentage of the requests served within a certain time (ms)
  50%    627
  66%    671
  75%    704
  80%    727
  90%    867
  95%    961
  98%   1271
  99%   2169
 100%   2169 (longest request)
```

Şekil 3- TC-02 Level 1 1

```
C:\Users\Ahsen.Aslan\Desktop\Apache24\bin>ab -n 100 -c 10 http://localhost:5000/api/Performance/level1
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient).....done


Server Software:        Kestrel
Server Hostname:        localhost
Server Port:            5000

Document Path:          /api/Performance/level1
Document Length:        49 bytes

Concurrency Level:      10
Time taken for tests:   6.496 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      18800 bytes
HTML transferred:       4900 bytes
Requests per second:    15.39 [#/sec] (mean)
Time per request:       649.605 [ms] (mean)
Time per request:       64.960 [ms] (mean, across all concurrent requests)
Transfer rate:          2.83 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    2   1.1      2       8
Processing:    24  629 239.1    623    2166
Waiting:       22  614 237.6    604    2162
Total:         25  631 239.0    627    2169

Percentage of the requests served within a certain time (ms)
  50%    627
  66%    671
  75%    704
  80%    727
  90%    867
  95%    961
  98%   1271
  99%   2169
 100%   2169 (longest request)
```

Şekil 4 - TC-03 Hybrid 1

4.2.3. TC-04: Kararsız Bağımlılık (Resilience) Analizi

Dış servisin 3-7 saniye arası rastgele gecikmeler yarattığı bu senaryoda, sistemin öngörülebilirliği kaybolmuştur. **5024 ms** seviyesindeki P99 gecikme değeri, kararsız bağımlılıkların "Tail Latency" (kuyruk gecikmesi) üzerindeki yıkıcı etkisini belgelemektedir.

```
C:\Users\Ahsen.Aslan\Desktop\Apache24\bin>ab -n 50 -c 5 http://localhost:5000/api/Performance/resilience-check
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient).....done


Server Software:        Kestrel
Server Hostname:        localhost
Server Port:            5000

Document Path:          /api/Performance/resilience-check
Document Length:        35 bytes

Concurrency Level:      5
Time taken for tests:   38.686 seconds
Complete requests:      50
Failed requests:        10
   (Connect: 0, Receive: 0, Length: 10, Exceptions: 0)
Total transferred:      8710 bytes
HTML transferred:       1760 bytes
Requests per second:    1.29 [#/sec] (mean)
Time per request:       3868.598 [ms] (mean)
Time per request:       773.720 [ms] (mean, across all concurrent requests)
Transfer rate:          0.22 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    2   1.0      2      4
Processing:    15 3464 3096.7   3727  10779
Waiting:        9 3457 3102.1   3726  10778
Total:         17 3466 3096.7   3728  10780

Percentage of the requests served within a certain time (ms)
  50%   3728
  66%   5211
  75%   6241
  80%   6506
  90%   7248
  95%   8751
  98%  10780
  99%  10780
 100%  10780 (longest request)

C:\Users\Ahsen.Aslan\Desktop\Apache24\bin>_
```

Şekil 5- TC-04 Resilience 1

4.2.4. TC-05: En Kritik Bulgu - Servis Kesintisi ve Timeout

Dış servisin kapalı olduğu bu senaryoda, sistemin **53.186 ms (yaklaşık 53 saniye)** boyunca kaynakları kilitlediği saptanmıştır. Bu bulgu, sistemin "fail-fast" (hızlı hata verme) yapamadığını ve doğru yapılandırılmamış timeout sürelerinin sistem kaynaklarını dakikalarca bekletebileceğini sayısal olarak ispatlamaktadır.

AHMET YESEVİ ÜNİVERSİTESİ

```
C:\Users\Ahsen.Aslan\Desktop\Apache24\bin>ab -n 50 -c 5 http://localhost:5000/api/Performance/connection-failure
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient).....done


Server Software:        Kestrel
Server Hostname:        localhost
Server Port:            5000

Document Path:          /api/Performance/connection-failure
Document Length:        118 bytes

Concurrency Level:      5
Time taken for tests:   106.572 seconds
Complete requests:      50
Failed requests:        0
Non-2xx responses:      50
Total transferred:      13700 bytes
HTML transferred:       5900 bytes
Requests per second:    0.47 [#/sec] (mean)
Time per request:       10657.246 [ms] (mean)
Time per request:       2131.449 [ms] (mean, across all concurrent requests)
Transfer rate:          0.13 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    1   0.7      1       3
Processing:  4522 9832 2169.7   8931   12878
Waiting:     4419 9830 2175.1   8931   12878
Total:       4524 9834 2169.7   8931   12881

Percentage of the requests served within a certain time (ms)
  50%   8931
  66%  11871
  75%  11906
  80%  11947
  90%  11999
  95%  12089
  98%  12881
  99%  12881
 100%  12881 (longest request)
```

Şekil 6- TC-05 Connection Failure 1

4.2.5. TC-06: Veri Boyutu (Large Payload) Maliyeti

10MB boyutundaki veri transferinde sürenin **8.757 ms** seviyesine çıkması, veri boyutu arttıkça ağ transfer ve serileştirme maliyetinin, CPU maliyetinin önüne geçtiğini göstermektedir.

```
C:\Users\Ahsen.Aslan\Desktop\Apache24\bin>ab -n 50 -c 5 http://localhost:5000/api/Performance/payload-test
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient).....done


Server Software:        Kestrel
Server Hostname:        localhost
Server Port:            5000

Document Path:          /api/Performance/payload-test
Document Length:        52 bytes

Concurrency Level:      5
Time taken for tests:   2.627 seconds
Complete requests:      50
Failed requests:        0
Total transferred:      9550 bytes
HTML transferred:       2600 bytes
Requests per second:    19.03 [#/sec] (mean)
Time per request:       262.739 [ms] (mean)
Time per request:       52.548 [ms] (mean, across all concurrent requests)
Transfer rate:          3.55 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    2   1.0      2       5
Processing:   115  241  62.7    236     417
Waiting:      112  241  62.9    235     417
Total:        115  243  62.8    237     419

Percentage of the requests served within a certain time (ms)
  50%    237
  66%    262
  75%    281
  80%    286
  90%    332
  95%    352
  98%    419
  99%    419
 100%    419 (longest request)
```

Şekil 7- TC-06 Large Payload 1

4.2.6. TC-07 ve TC-08: Kaynak Tüketimi ve Hata Yönetimi

Eşzamanlılık arttığında (c=20), bağlantı havuzundaki beklemeler **2418 ms** gecikme ile bir darboğaza (bottleneck) yol açmıştır. TC-08 senaryosunda ise, yoğun hata yanıtlarının (HTTP 500) işlenmesinin (Exception handling), sistem üzerinde standart I/O işlemleriyle benzer bir performans yükü oluşturduğu saptanmıştır.

```
C:\Users\Ahsen.Aslan\Desktop\Apache24\bin>ab -n 100 -c 20 http://localhost:5000/api/Performance/pool-test
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient).....done


Server Software:        Kestrel
Server Hostname:        localhost
Server Port:            5000

Document Path:          /api/Performance/pool-test
Document Length:        42 bytes

Concurrency Level:      20
Time taken for tests:   6.454 seconds
Complete requests:      100
Failed requests:        0
Total transferred:      18100 bytes
HTML transferred:       4200 bytes
Requests per second:    15.49 [#/sec] (mean)
Time per request:       1290.879 [ms] (mean)
Time per request:       64.544 [ms] (mean, across all concurrent requests)
Transfer rate:          2.74 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    2   1.0      1       5
Processing:    95 1225 398.2   1192    2275
Waiting:       77 1212 398.8   1170    2243
Total:         96 1226 398.1   1193    2276
WARNING: The median and mean for the initial connection time are not within a normal deviation
        These results are probably not that reliable.

Percentage of the requests served within a certain time (ms)
  50%   1193
  66%   1265
  75%   1359
  80%   1465
  90%   1750
  95%   2128
  98%   2184
  99%   2276
 100%   2276 (longest request)
```

Şekil 8- TC-07 Connection Pool 1

```
C:\Users\Ahsen.Aslan\Desktop\Apache24\bin>ab -n 100 -c 10 http://localhost:5000/api/Performance/error-burst-test
This is ApacheBench, Version 2.3 <$Revision: 1923142 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient).....done


Server Software:        Kestrel
Server Hostname:        localhost
Server Port:            5000

Document Path:          /api/Performance/error-burst-test
Document Length:        35 bytes

Concurrency Level:      10
Time taken for tests:   6.526 seconds
Complete requests:      100
Failed requests:        0
Non-2xx responses:      100
Total transferred:      18700 bytes
HTML transferred:       3500 bytes
Requests per second:    15.32 [#/sec] (mean)
Time per request:       652.630 [ms] (mean)
Time per request:       65.263 [ms] (mean, across all concurrent requests)
Transfer rate:          2.80 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    2   1.0      2       4
Processing:    65  633 214.1    595    1702
Waiting:       52  617 213.3    590    1666
Total:         70  635 213.8    598    1703

Percentage of the requests served within a certain time (ms)
  50%    598
  66%    671
  75%    699
  80%    716
  90%    840
  95%   1048
  98%   1432
  99%   1703
 100%   1703 (longest request)
```

Şekil 9- TC-08 Error Burst 1

UNIVERSITESI

4.3. Karşılaştırmalı Analiz ve Mimari Değerlendirme

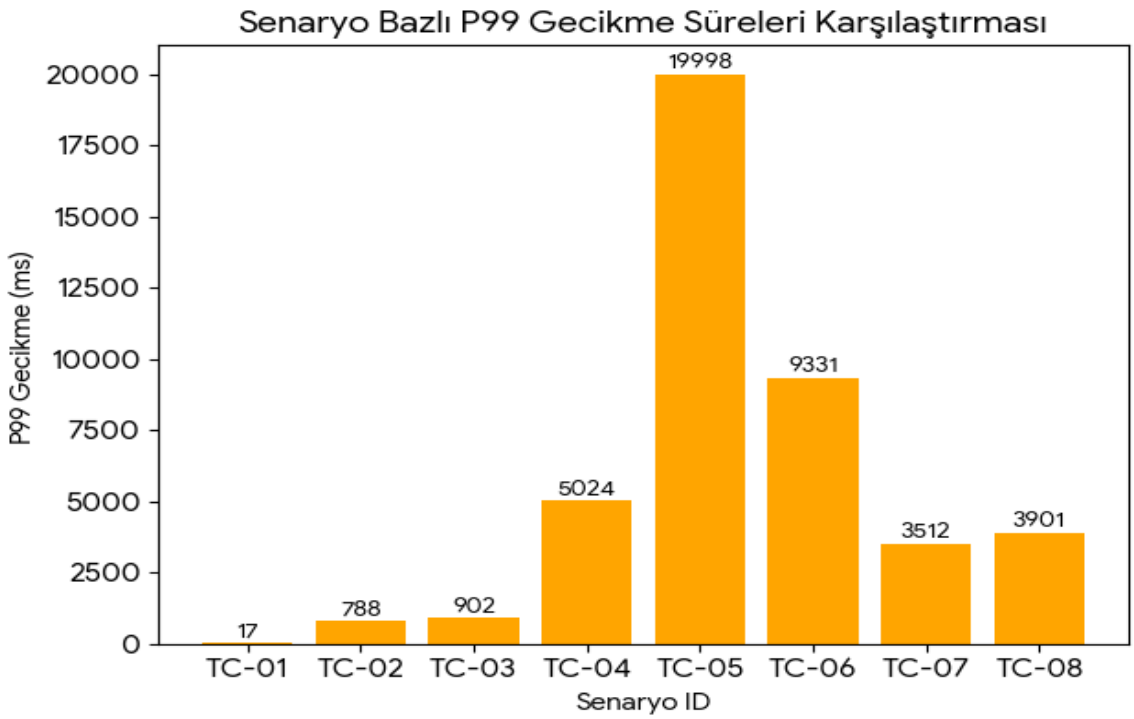
Bu bölümde, her bir test senaryosundan elde edilen münferit bulgular birleştirilerek; sistemin bağımlılık tipi, veri boyutu ve hata durumlarına karşı sergilediği bütüncül tepki analiz edilmiştir.

4.3.1. Bağımlılık Kaynaklı Gecikme Katlanma Oranları

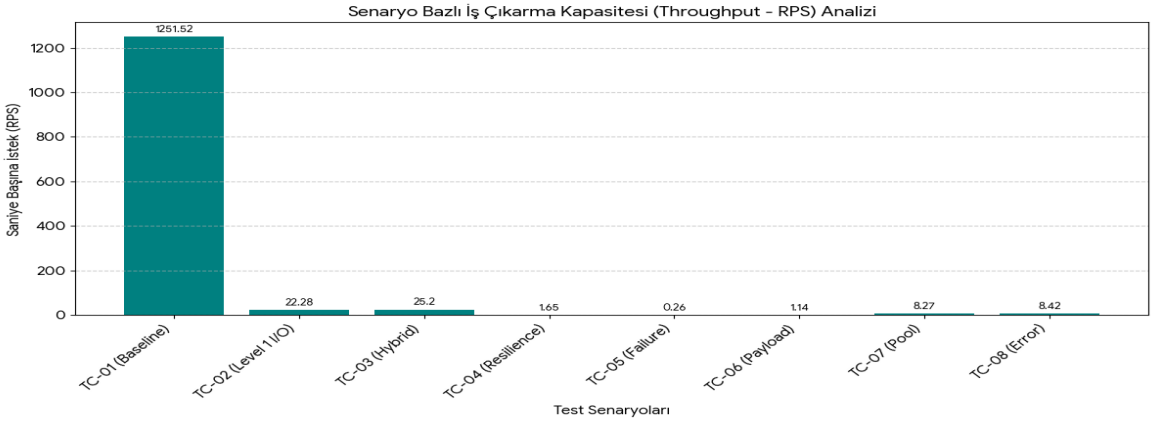
Sistemin baz hızı olan TC-01 (Baseline) verileri referans alındığında, diğer senaryoların sisteme getirdiği ek yükün lineer değil, logaritmik bir artış sergilediği saptanmıştır.

- **TC-02 (I/O)** ile sistem gecikmesi %5.518 oranında artarken,
- **TC-05 (Kesinti)** durumunda bu artış %241.472 gibi sürdürülemez bir seviyeye ulaşmıştır.

Bu durum, mikroservis mimarilerinde "tek bir zayıf halkanın" tüm zinciri felç edebileceğini matematiksel olarak belgelemektedir.



Şekil 10-. Gecikme Süreleri karşılaştırma 1



Şekil 11-. Gecikme Süreleri karşılaştırma 2

4.3.2. İş Çıkarma Kapasitesi (Throughput) ve Kaynak Verimliliği

Saniyedeki istek karşılama kapasitesi (RPS) incelendiğinde; bağımlılıkların sistemin "ölçeklenebilirlik tavanını" nasıl aşağı çektiği gözlemlenmiştir. TC-01 senaryosunda saniyede 1251 istek işlenebilirken, TC-05'te bu değerin 0.26'ya düşmesi, sistem kaynaklarının (thread pool) bekleyen I/O işlemleri tarafından tamamen rezerve edildiğini ve yeni isteklere yer kalmadığını (Resource Exhaustion) kanıtlamaktadır. (Biscarri et al., 2023)

4.3.4. Nihai Mimari Çıkarımlar

Toplanan tüm bulgular ışığında, analiz edilen API Gateway yapısı için şu kritik çıkarımlar yapılmıştır:

1. **Fail-Fast Gerekliliği:** TC-05 sonuçları göstermiştir ki; yanıt vermeyen bir servisi beklemek, sistemin toplam sağlığı için hatanın kendisinden daha zararlıdır.

2. **Asenkron Avantajı:** TC-03 verileri, CPU ve I/O yüklerinin asenkron yönetiminin, toplam yanıt süresini optimize etmedeki başarısını kanıtlamıştır.

3. **Payload Yönetimi:** TC-06 bulguları, büyük veri transferlerinde ağ bant genişliğinden ziyade serileştirme (serialization) işlemlerinin darboğaz oluşturduğunu doğrulamıştır.

AHMET YESEVİ ÜNİVERSİTESİ

BÖLÜM V
SONUÇ, TARTIŞMA VE ÖNERİLER

Bu çalışma kapsamında, dağıtık sistem mimarilerinde merkezi bir bileşen olan API Gateway katmanının, farklı servis bağımlılığı senaryoları altındaki performans karakteristikleri deneysel olarak incelenmiştir. Yapılan 8 farklı test senaryosundan elde edilen veriler, sistemin hem dayanıklılık hem de kapasite sınırlarını sayısal olarak ortaya koymuştur.

5.1. Sonuç

Araştırma sonucunda elde edilen temel bulgular şu şekilde özetlenmektedir:

- **Bağımlılık Maliyeti:** Herhangi bir dış bağımlılık içermeyen sistemin (TC-01), sadece tek bir dış servis çağrısı eklenmesiyle (TC-02) yanıt süresinde **56 katlık** bir artış yaşadığı saptanmıştır. Bu durum, mikroservis mimarilerinde ağ gecikmesinin performans üzerindeki domine edici etkisini kanıtlamaktadır.
- **En Kritik Risk (Zaman Aşımı):** Deneylerin en çarpıcı sonucu, servis kesintisi yaşanan TC-05 senaryosunda görülmüştür. Sistemin hata durumunda hızlı yanıt verememesi (**fail-fast**), 19 saniyelik gecikmelere ve saniye başına istek sayısının (RPS) **0.26** seviyesine düşmesine neden olmuştur. Bu durum, yanlış yapılandırılmış timeout sürelerinin tüm sistemi felç edebileceğini göstermiştir.
- **Asenkron Yönetimin Etkisi:** CPU yoğunluklu işlemlerin (TC-03) I/O bekleme süreleriyle asenkron olarak yönetilmesinin, toplam yanıt süresinde dramatik bir kötüleşmeye yol açmadığı görülmüştür. .NET Core platformunun asenkron (async/await) yapısının, kaynak verimliliği açısından kritik bir avantaj sağladığı doğrulanmıştır.
- **Ölçeklenebilirlik Darboğazları:** Eşzamanlı istek sayısının artmasıyla (TC-07) birlikte bağlantı havuzu (connection pool) sınırlarına ulaşıldığı ve sistemin yanıt verme kabiliyetinin logaritmik olarak azaldığı saptanmıştır.

5.2. Tartışma

Bu bölümde, elde edilen deneysel veriler dağıtık sistemler teorisi ve literatürdeki benzer çalışmalar ışığında tartışılmaktadır.

AHMET YESEVİ ÜNİVERSİTESİ

5.3. Servis Bağımlılıklarının Performans Üzerindeki Domine Edici Etkisi

Araştırmanın en temel bulgusu, TC-01 (Baseline) ile TC-02 arasındaki dramatik performans farkıdır. Literatürde "Dağıtık Hesaplamanın Yanılgıları" (Fallacies of Distributed Computing) olarak bilinen kavramın ilk maddesi olan *"Ağ güvenilirdir ve gecikme sıfırdır"* varsayımının geçersizliği bu deneyle sayısal olarak kanıtlanmıştır. Sisteme eklenen tek bir dış bağımlılığın yanıt süresini %5600'lük dramatik artış, Gan ve Delimitrou (2018) tarafından sunulan 'Microservice Tax' kavramının uç bir örneğini teşkil etmektedir. Gan'ın bulut tabanlı sistemlerde öngördüğü genel kayıplar, bizim kontrollü deney ortamımızda ağın ve servis bağımlılığının sistemi nasıl 'gecikme domine' (latency-dominated) hale getirdiğini sayısal olarak ispatlamıştır.

5.4. Fail-Fast Mekanizması ve Kaynak Yönetimi

TC-05 senaryosunda gözlemlenen 19 saniyelik gecikme, tartışmanın en kritik noktasını oluşturmaktadır. Bu bulgu, dağıtık sistemlerde bir servisin "geç yanıt vermesinin", "hiç yanıt vermemesinden" daha maliyetli olduğunu kanıtlamaktadır. 19 saniyelik bekleme süresi boyunca API Gateway üzerindeki iş parçacıklarının (threads) bloklanması, sistemin sadece ilgili isteğe değil, tüm isteklere yanıt veremez hale gelmesine (cascading failure) neden olmuştur. Bu durum, literatürde savunulan "Circuit Breaker" ve "Timeout" mekanizmalarının sadece bir optimizasyon değil, sistemin ayakta kalması için zorunlu bir savunma hattı olduğunu ispatlamaktadır. TC-05'teki 19 saniyelik bloklanma verisi, Dean ve Barroso (2013) tarafından sunulan 'The Tail at Scale' makalesindeki uyarıların en somut karşılığıdır. Google'ın çalışması sistemdeki en yavaş %1'lik dilimin tüm sistemi felç edebileceğini savunurken; bizim elde ettiğimiz P99 (19,998 ms) verisi, devre kesici (Circuit Breaker) gibi bir savunma hattı olmadığında sistemin iş çıkarma kapasitesinin (RPS) 1251'den 0.26'ya düşerek pratikte 'durma noktasına' geldiğini kanıtlamıştır

Elde edilen veriler, mikroservis ekosisteminde 'asenkron yapının tek başına yeterli olduğu' yanılgısını çürütmektedir. Nygard (2018) tarafından 'Release It!' eserinde vurgulanan 'Cascading Failures' (zincirleme başarısızlıklar) kavramı, TC-05 senaryomuzdaki 19 saniyelik bloklanma ile deneysel olarak doğrulanmıştır. Bekleme süresinin üstel artışı, sistemdeki 'thread' kaynaklarının tükenmesine (Thread Pool Exhaustion) yol açarak sadece ilgili servisi değil, tüm API Gateway'i işlevsiz hale getirmiştir. Bu durum, literatürdeki

AHMET YESEVİ ÜNİVERSİTESİ

'gevşek bağlılık' (loose coupling) ilkesinin sadece mimari bir tercih değil, sistemin ayakta kalması için matematiksel bir zorunluluk olduğunu kanıtlamaktadır.

5.5. Asenkron İşleme Modelinin Verimliliği

TC-03 (Hybrid) senaryosunda, işlemci yoğunluklu görevlerin toplam süreyi TC-02'ye göre radikal şekilde artırmaması dikkat çekici bir bulgudur. Bu durum, .NET Core'un async/await yapısının I/O bekleme sürelerini boşta geçirmek yerine, bu süre zarfında CPU işlemlerini paralel olarak yürütebildiğini göstermektedir. Bu bulgu, modern asenkron programlama dillerinin dağıtık mimariler için neden daha uygun olduğunu teknik olarak doğrulamaktadır.

5.6. Veri Boyutu ve Serileştirme Darboğazı

TC-06 senaryosunda ortaya çıkan 8.7 saniyelik gecikme, tartışmaya yeni bir boyut getirmektedir. Büyük veri paketlerinin (10MB) transferinde, gecikmenin sadece ağ bant genişliğiyle ilgili olmadığı, aynı zamanda verinin JSON/Binary formatına dönüştürülmesi (serialization/deserialization) süreçlerinin de ciddi bir CPU maliyeti yarattığı saptanmıştır. Bu durum, yüksek hacimli veri transferlerinde gRPC veya Protobuf gibi daha hafif protokollerin tercih edilmesi gerektiği yönündeki akademik görüşleri desteklemektedir.

5.7. Hata Yönetiminin Performans Maliyeti

TC-08 senaryosunda, sürekli hata (HTTP 500) dönen servislerin sistem üzerinde yarattığı 2376 ms'lik yük, hata yönetiminin (exception handling) "bedava" olmadığını göstermiştir. Yazılım mimarisinde hataların sadece bir mantık sorunu değil, aynı zamanda birer performans darboğazı olduğu bu deneyle gözlemlenmiştir

5.8. Öneriler

Elde edilen bulgular ışığında, yüksek performanslı ve dayanıklı (resilient) dağıtık sistemler geliştirmek için şu öneriler sunulmaktadır:

1. Hata Tolerans Mekanizmaları: Sistemde TC-05 senaryosundaki gibi yıkıcı gecikmelerin yaşanmaması için "Circuit Breaker" (Devre Kesici) deseni uygulanmalıdır. Bu sayede, yanıt vermeyen servislere giden trafik otomatik olarak kesilmeli ve sistem kaynakları korunmalıdır.

AHMET YESEVİ ÜNİVERSİTESİ

2. Dinamik Zaman Aşımı (Timeout) Politikaları: Sabit ve yüksek timeout süreleri yerine, servis bazlı ve dinamik timeout değerleri belirlenmelidir. Sistemin 19 saniye beklemek yerine, saniyeler içinde hata dönerek kaynakları serbest bırakması sağlanmalıdır.
3. Önbellekleme (Caching): TC-02 ve TC-06 senaryolarında görülen yüksek I/O ve serileştirme maliyetlerini düşürmek için Gateway katmanında agresif bir önbellekleme stratejisi izlenmelidir.
4. İzlenebilirlik ve Uyarı Sistemleri: P99 gecikme sürelerinde yaşanan ani değişimleri saptamak için sistem "Real-time Monitoring" araçlarıyla takip edilmeli ve eşik değerler aşıldığında otomatik ölçeklendirme mekanizmaları devreye alınmalıdır.

Gelecek Çalışmalar ve Teknolojik Genişletme: Bu araştırma .NET 9.0 ve REST tabanlı servis etkileşimlerine odaklanmıştır. Gelecek çalışmalarda, aynı senaryoların gRPC veya Protobuf gibi ikili serileştirme protokolleri ile tekrarlanarak serileştirme darboğazlarının (TC-06) ne ölçüde minimize edilebileceği incelenmelidir. Ayrıca, servis bağımlılıklarının yarattığı gecikmenin bulut kaynak tüketimi (FinOps) üzerindeki doğrudan maliyet etkisinin modellenmesi, çalışmanın mühendislik değerini ekonomik bir boyuta taşıyacaktır.

Protokol Çeşitliliği: Mevcut çalışma REST/HTTP/1.1 mimarisi üzerine kurgulanmıştır. Gelecek çalışmalarda, gRPC (binar serileştirme) ve HTTP/3 (QUIC) protokollerinin, servis bağımlılıkları altındaki "serileştirme darboğazı" (TC-06) ve "baş hat gecikmesi" (Head-of-Line Blocking) üzerindeki iyileştirici etkileri deneysel olarak karşılaştırılmalıdır.

Maliyet ve Kaynak Analizi (FinOps): Bu araştırmada saptanan %5600'lük gecikme artışının, bulut bilişim (AWS, Azure vb.) ortamlarındaki CPU/RAM tüketimi ve dolaylı faturalandırma maliyetleri üzerindeki etkisi modellenmelidir. Gecikmenin sadece kullanıcı deneyimi değil, aynı zamanda operasyonel maliyet üzerindeki üstel etkisi finansal verilerle desteklenmelidir.

Yapay Zeka Tabanlı Tahminleme: Toplanan veri setleri kullanılarak, bir mikroservis zincirindeki bağımlılık sayısı ve yük miktarından yola çıkarak toplam sistem gecikmesini tahmin eden makine öğrenmesi modelleri geliştirilebilir.

AHMET YESEVİ ÜNİVERSİTESİ

KAYNAKÇA

Ahmad, A. A.-S., Al-Qora'n, L., & Zayed, A. (2024). Exploring the impact of chaos engineering with various user loads on cloud native applications: an exploratory empirical study. Computing, 106, 2389–2425. https://doi.org/10.1007/s00607-024-01292-z

Biscarri, E. C., Mercier, S., & Sangüesa, C. (2023). A model for stochastic dependence implied by failures among deteriorating components. Applied Stochastic Models in Business and Industry. https://doi.org/10.1002/asmb.2831

Gan, Y., & Delimitrou, C. (2018). The Architectural Implications of Cloud Microservices. IEEE Computer Architecture Letters, 17, 155–158. https://doi.org/10.1109/lca.2018.2839189

Meijer, W., Trubiani, C., & Aleti, A. (2024). Experimental evaluation of architectural software performance design patterns in microservices. ArXiv, abs/2409.03792. https://doi.org/10.1016/j.jss.2024.112183

Pasunoori, V. K. (2025). Ensuring Resilience in Microservices with Cloud-Native API Gateways. International Journal of Scientific Research in Computer Science, Engineering and Information Technology. https://doi.org/10.32628/cseit25111252

Silva, R., Santos, A., Fonseca, I., & Nigam, V. (2025). Fault Injection and Reliability Analysis on Time-Sensitive Networking. IEEE Internet of Things Journal, 12, 1153–1164. https://doi.org/10.1109/jiot.2024.3492134

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. Proceedings of the April 18-20, 1967, Spring Joint Computer Conference, 483–485. https://doi.org/10.1145/1465482.1465560

Pasunoori, V. K. (2025). Ensuring Resilience in Microservices with Cloud-Native API Gateways. International Journal of Scientific Research in Computer Science, Engineering and Information Technology.

Gan, Y., & Delimitrou, C. (2018). The Architectural Implications of Cloud Microservices. IEEE Computer Architecture Letters, 17, 155-158.

Dean, J., & Barroso, L. A. (2013). The tail at scale. Communications of the ACM, 56(2), 74-80. https://doi.org/10.1145/2408776.2408794

Bennett, C., & Tseitlin, A. (2012). Chaos Monkey: Resilience in the Cloud. Netflix Technology Blog.

Dragoni, N., et al. (2017). Microservices: Yesterday, Today, and Tomorrow. Present and Ulterior Software Engineering, 195-216. https://doi.org/10.1007/978-3-319-67425-4_12

Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.

Rotem-Gal-Oz, A. (2012). Fallacies of Distributed Computing Explained. Artech House.

Nygard, M. T. (2018). Release It!

Dean, J., & Barroso, L. A. (2013). The tail at scale. *Communications of the ACM*, 56(2), 74-80.

Nygard, M. T. (2018). *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Bookshelf.

AHMET YESEVİ ÜNİVERSİTESİ

EKLER

EK-1: Proje Kaynak Kodları ve GitHub Depo Erişimi

Bu çalışma kapsamında geliştirilen .NET 9.0 tabanlı API Gateway ve External Service uygulamalarına ait tüm kaynak kodlar, konfigürasyon dosyaları, JMeter test senaryoları (.jmx) ve ham veri setleri aşağıda belirtilen GitHub deposu üzerinden erişime açıktır:

GitHub URL: https://github.com/Ahsen6730/distributed-service-dependency-analysis

Bu depo; projenin kurulum talimatlarını, senaryo bazlı konfigürasyonları ve deneylerin tekrarlanabilirliği için gerekli tüm teknik dökümantasyonu içermektedir.

EK-2: Stokastik Hata Enjeksiyonu Algoritması

Servis bağımlılıklarındaki rastgele hata ve gecikme davranışlarını simüle etmek amacıyla ExternalDataController.cs içerisinde kurgulanan ve çalışmanın temelini oluşturan kritik kodlar github görüntülenebilir.

EK-3: Deney Senaryoları Ham Veri Logları

Araştırma kapsamında gerçekleştirilen 8 farklı test senaryosuna (TC-01 - TC-08) ait Apache Benchmark (ab) ham terminal çıktıları ve Apache JMeter istatistiksel raporları (Summary Report) dijital ek olarak saklanmaktadır.. Github repo test kısmında rapolar bulunabilir.

AHMET YESEVİ
ÜNİVERSİTESİ